



**GOVERNO DO ESTADO
DE SÃO PAULO**

PROGRAMAÇÃO WEB
PESQUISA SOBRE ARQUITETURA DE SOFTWARE

Hugo Florio Iorio – RA 0030482511024

Sorocaba
Agosto/2026

SUMÁRIO

1. Introdução.....	3
2. Conceitos.....	3
3. Padrões.....	4
4. Escolhas.....	4
5. Tendências.....	5
6. Conclusão.....	5
7. Referências.....	6

1. Introdução

A arquitetura de software desempenha um papel fundamental no ciclo de vida de sistemas computacionais, atuando como a fundação estrutural sobre a qual todas as funcionalidades, requisitos não funcionais e evoluções do sistema são construídos. Em um cenário tecnológico dinâmico, onde a demanda por soluções ágeis e escaláveis é constante, compreender as decisões de design e a organização dos componentes torna-se essencial para garantir a qualidade, o desempenho e a sustentabilidade dos sistemas ao longo do tempo.

Este trabalho tem como objetivo apresentar uma visão conceitual e prática sobre a arquitetura de software, abordando seus conceitos fundamentais, principais elementos constitutivos e os padrões arquiteturais predominantes. Além disso, analisa-se a relevância do processo contínuo de tomada de decisão na escolha de abordagens estruturais, bem como as principais tendências modernas que vêm moldando o ecossistema de desenvolvimento de software, como os modelos em nuvem e a integração de tecnologias de inteligência artificial.

2. Conceitos

Arquitetura de software pode ser definida como um conjunto de decisões de design tomadas durante o planejamento e desenvolvimento de um software, tendo como propósito proporcionar maior funcionalidade, escalabilidade e manutenção, servindo essencialmente como a “espinha dorsal” do desenvolvimento (SU et al., 2025).

Neste sentido, Medvidovic e Taylor (2010) descrevem alguns conceitos importantes:

- Componentes: Unidades de computação ou abstração do sistema;
- Conectores: Mediadores das interações entre os componentes;
- Configurações (Ou relacionamentos): Arranjos entre componentes e conectores, junto das regras que regem suas interações;
- Escalabilidade: Capacidade do sistema de crescer;
- Disponibilidade: Capacidade do sistema de permanecer acessível aos seus usuários;
- Segurança: Proteção de dados ou recursos do sistema;
- Confiabilidade: Garantia de que o sistema irá operar como esperado;
- Manutenibilidade: Facilidade com que o sistema pode ser modificado após seu lançamento.

3. Padrões

Na prática da arquitetura de software, padrões de arquitetura podem ser descritos como sendo soluções universais e reutilizáveis para resolver problemas comuns (BUSCHMANN; HENNEY, 2008). Também podem ser chamados de “estilos arquiteturais”, descrevendo esquemas de organização fundamentais para o software.

Dos padrões de arquitetura comumente utilizados, Farshidi, Jansen e Van Der Werf (2020) descrevem:

- Camadas: Organiza o sistema em diferentes níveis de abstração, focando em reusabilidade, mas podendo enfrentar desafios com escalabilidade;
- Cliente-Servidor: Estrutura na qual um cliente solicita recursos e um servidor os fornece;
- Tubos e Filtros: Arquitetura focada no fluxo de dados através de passos independentes (filtros) conectados por canais de comunicação (tubos);
- Model-View-Controller: Visando facilitar o design de aplicações, divide seu desenvolvimento em três partes, que são o modelo (dados e lógica), visão (interface de usuário) e o controlador (intermediário entre lógica e interface de usuário);
- Arquitetura Monolítica: Arquitetura mais tradicional, é caracterizada como sistemas que reúnem, em uma única unidade, os diversos componentes presentes no software;
- Arquitetura de Microsserviços: Arquitetura mais complexa que se utiliza de recursos fracamente acoplados, mas que têm menor dependência entre si, tendo melhor escalabilidade e resiliência.

4. Escolhas

As escolhas tomadas na arquitetura de software têm grande importância no andamento e resultado de projetos, pois definem quais serão os componentes, os recursos e como eles serão utilizados, ditando essencialmente como será sua funcionalidade e evolução (MEDVIDOVIC; TAYLOR, 2010). O processo de tomada de decisões é contínuo durante o desenvolvimento do software, de forma que o arquiteto deve avaliar as diferentes alternativas de acordo com o que considera ideal para o projeto (FARSHIDI; JANSEN; VAN DER WERF, 2020).

Diferentes arquiteturas têm diferentes pontos fortes e diferentes fraquezas. A arquitetura de microsserviços, por exemplo, destaca-se pela forte escalabilidade e flexibilidade dos componentes, mas também apresenta alta complexidade operacional, principalmente na consistência de dados

(AGWENYI; MBUGUA, 2025). Por outro lado, arquiteturas monolíticas, com forte acoplamento, podem ser muito mais simples em comparação, mas podem sofrer com escalabilidade limitada, visto que modificar um dos componentes pode comprometer o funcionamento de outros componentes.

5. Tendências

As tendências modernas em arquitetura de software são lideradas pelas abordagens cloud-native e serverless, que se tornaram modelos centrais da infraestrutura tecnológica atual ao focar em escalabilidade e flexibilidade. O design cloud-native utiliza containers e ferramentas de orquestração como o Kubernetes (ferramenta de código aberto para gerenciamento de aplicações em contêiner) para garantir alta disponibilidade e resiliência, permitindo uma entrega de software mais rápida (SU et al., 2025).

Complementarmente, o modelo serverless permite que desenvolvedores criem e implantem aplicações sem a necessidade de gerenciar servidores, utilizando um sistema de custo por uso e escalonamento automático, o que reduz custos operacionais e simplifica o processo de desenvolvimento (AGWENYI; MBUGUA, 2025).

Além disso, a recente popularização de modelos de inteligência artificial também apresentou uma nova abordagem à arquitetura de software, integrando ferramentas de geração e análise para automatizar tarefas, agilizando o desenvolvimento e aumentando a produtividade, embora ainda existam desafios e implicações éticas (AGWENYI; MBUGUA, 2025).

6. Conclusão

Diante do estudo realizado, constata-se que a arquitetura de software não é apenas uma etapa preliminar de modelagem, mas sim uma disciplina essencial que direciona a viabilidade técnica e operacional de um sistema. A escolha adequada de conceitos, componentes e padrões arquiteturais impacta diretamente a capacidade de escalabilidade, manutenção, resiliência e segurança do software em longo prazo.

Por fim, observa-se que o avanço contínuo das infraestruturas baseadas em nuvem e a incorporação de inteligência artificial vêm redefinindo os papéis e os desafios na engenharia de software. Compreender o alinhamento entre as necessidades do negócio, as trocas entre benefícios e malefícios operacionais e as tendências emergentes é indispensável para que arquitetos e desenvolvedores possam projetar soluções robustas, adaptáveis e alinhadas às crescentes demandas do mercado de tecnologia.

7. Referências

AGWENYI, C. A.; MBUGUA, S. M. **Trends in Software Architecture Designs: Evolution and Current State**. International Journal of Innovative Science and Research Technology, v. 10, n. 3, p. 1725-1729, mar. 2025. DOI: <https://doi.org/10.38124/ijisrt/25mar1311>

BUSCHMANN, Frank; HENNEY, Kevlin. **Pattern-Oriented Software Architecture: The Craft of Software Architecture**. Munich: Siemens AG / Curbralan Limited, 2008. Presentation slides from OOP 2008, Munich, Germany, 21-25 Jan. 2008. Disponível em: https://www.researchgate.net/publication/228722744_Pattern-oriented_software_architecture

FARSHIDI, Siamak; JANSEN, Slinger; VAN DER WERF, Jan Martijn. **Capturing software architecture knowledge for pattern-driven design**. The Journal of Systems & Software, v. 169, p. 110714, nov. 2020. DOI: <https://doi.org/10.1016/j.jss.2020.110714>.

MEDVIDOVIC, Nenad; TAYLOR, Richard N. **Software Architecture: Foundations, Theory, and Practice**. In: INTERNATIONAL CONFERENCE ON SOFTWARE ENGINEERING (ICSE), 32., 2010, Cape Town. Proceedings [...]. Cape Town: ACM, 2010. p. 471-472. DOI: <https://doi.org/10.1145/1810295.1810433>.

SU, Ruoyu et al. **Emerging Trends in Software Architecture from the Practitioner's Perspective: A Five-Year Review**. *arXiv preprint arXiv:2507.14554v2*, 25 jul. 2025. Disponível em: <https://arxiv.org/abs/2507.14554v2>.